

COMPUTER ORGANIZATION AND ARCHITECTURE

UNIT –II

TOPIC- GENERAL REGISTER ORGANIZATION

General Register Organization

Memory locations are needed for storing pointers, counters, return addresses, temporary results and partial products during operations. Having to refer to memory locations is time consuming because memory access is the most time - consuming operation in a computer.

It is more convenient and more efficient to store these intermediate values in processor registers.

When a large number of registers are included in the CPU, it is most efficient to connect them through a common bus system. The registers communicate with each other not only for direct data transfers, but also while performing various micro-operations.

A bus organization for seven CPU registers is as shown below:

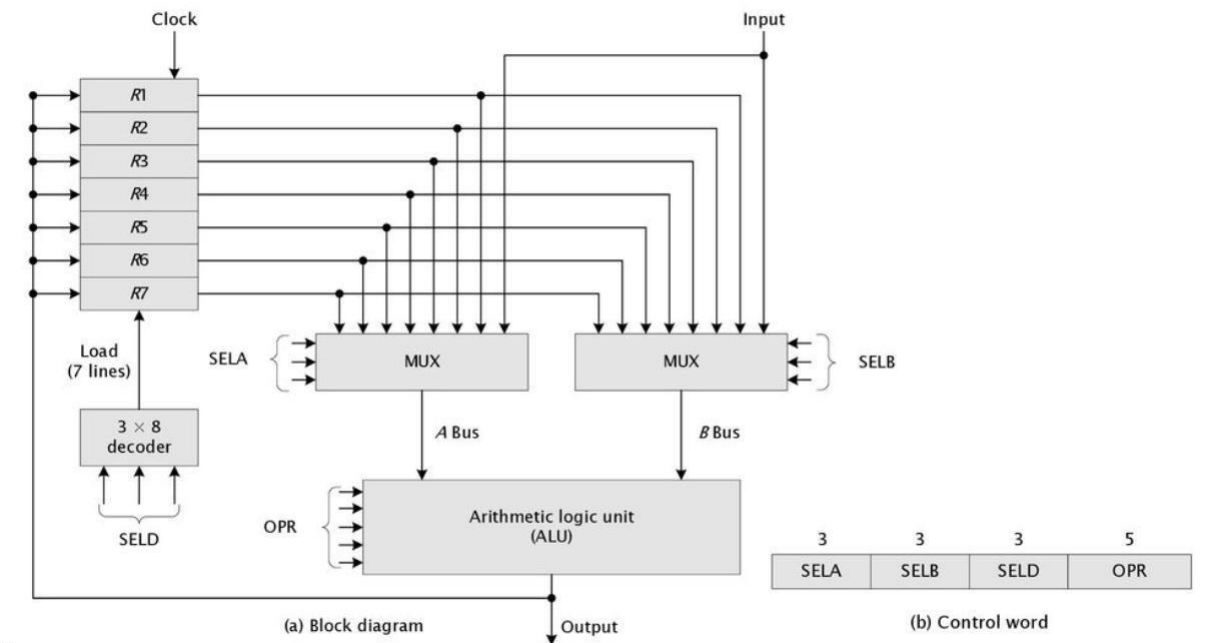


Fig: General Register Organization

The output of each register is connected to two multiplexers (MUX) to form the two buses A and B.

- The selection lines in each multiplexer select one register or the input data for the particular bus.

- The A and B buses form the inputs to a common arithmetic logic unit (ALU).
- The operation selected in the ALU determines the arithmetic or logic micro- operation that is to be performed.
- The result of the micro-operation is available for output data and also goes into the inputs of all the register.
- The register that received the information from the output bus is selected by a decoder. The decoder activates one of the register load inputs, thus providing a transfer path between the data in the output bus and the inputs of the selected destination register.

The control unit that operates the CPU bus system directs the information flow through the registers and ALU by selecting the various components in the system.

Control Word: There are 14 binary selection inputs in the unit, and their combined value specifies a control word.

It consists of four fields:

- The three bits of SELA select a source register for the A input of the ALU.
- The three bits of SELB select a register for the B input of the ALU.
- The three bits of SELD select a destination register using the decoder and its seven load outputs.
- The five bits of OPR select one of the operations in ALU.

The 14-bit control word when applied to the selection inputs specify a particular micro-operation.

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

Table: Encoding of Register Selection Fields

The 3-bit binary code listed in the first column of the table specifies the binary code for each of the three fields.

The register selected by fields SELA, SELB and SELD is the one whose decimal number is equivalent to the binary number in the code.

When SELA or SELB is 000, the corresponding multiplexer selects the external input data.

When SELD=000, no destination register is selected but the contents of the output bus are available in the external output.

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add A + B	ADD
00101	Subtract A – B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

Table: Encoding of ALU Operations

The ALU provides arithmetic and logic operations.

The function table for this ALU is shown in above table.

The OPR field has five bits and each operation is designated with a symbolic name.

Example: To perform the operation: $R1 \leftarrow R2 + R3$

The control must provide binary selection variables to the following selector inputs:

1. MUX A selector (SELA): to place the content of R2 into bus A.
2. MUX B selector (SELB): to place the content of R3 into bus B.
3. ALU operation selector (OPR): to provide the arithmetic addition A+B.
4. Decoder destination selector (SELD): to transfer the content of the output bus into R1.

A control word of 14 bits is needed to specify this micro-operation.

Field:	SELA	SELB	SELD	OPR
Symbol:	R2	R3	R1	ADD
Control Word	010	011	001	00010

COMPUTER ORGANIZATION AND ARCHITECTURE

UNIT –II

TOPIC- INSTRUCTION FORMATS,

TYPES OF INSTRUCTION FORMATS WITH EXAMPLE PART-1

Instruction Formats

An instruction format refers to the layout or structure of a machine-level instruction, which consists of several fields that hold different types of information. Each field in an instruction has a specific role in telling the CPU what operation to perform, where the operands are located, and how to access them.

The typical components of an instruction format include:

- **Opcode:** The operation to be performed (e.g., addition, subtraction).
- **Operands:** The data or addresses involved in the operation.
- **Addressing Modes:** It defines how the operands are accessed (e.g., direct, indirect).

It can be of variable lengths with multiple numbers of addresses. These address fields in the instruction format vary as per the organization of the registers in the CPU. The formats supported by the CPU depend upon the Instructions Set Architecture implemented by the processor.

Most of the computers fall into one of three types of CPU organizations:

1. Single accumulator organization.
2. General register organization.
3. Stack organization.

1. Single Accumulator Organization

All the operations on a system are performed with an implied accumulator register. The instruction format in this type of computer uses one address field.

For example, the instruction for arithmetic addition is defined by an assembly language instruction 'ADD.'

$$AC \leftarrow AC + M[X].$$

Where X is the operand's address, the ADD instruction results in the operation
AC is the accumulator register, M[X] symbolizes the memory word located at address X.

2. General Register Organization

The general register type computers employ two or three address fields in their instruction format. Each address field specifies a processor register or a memory.

An instruction symbolized by ADD R1, X specifies the operation

$$R1 \leftarrow R + M[X].$$

It has two address fields: register R1 and memory address X.

3. Stack Organization

A computer with a stack organization has PUSH and POP instructions that require an address field.

Hence, the instruction PUSH X pushes the word at address X to the top of the stack. The stack pointer updates automatically.

In stack-organized computers, the operation type instructions don't require an address field as the operation is performed on the two items on the top of the stack.

Types of Instruction formats

Depending on the multiple address fields, the instruction is categorized as follows:

1. **Zero address instruction**
2. **One address instruction**
3. **Two address instruction**
4. **Three address instruction**

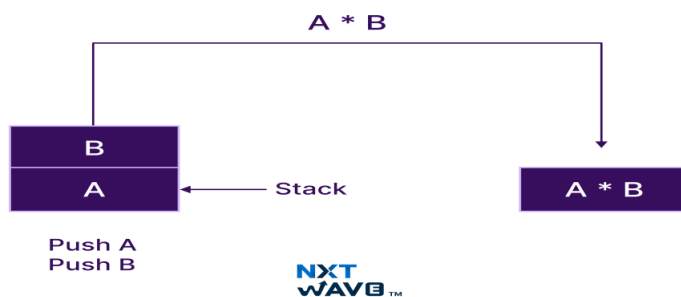
The operations specified by a computer instruction are executed on data stored in memory or processor registers. The operands residing in processor registers are specified with an address. The registered address is a binary number of k bits that defines one of the 2^k registers in the CPU.

Thus, a CPU with 16 processors registers R0 through R15 and will have a four-bit register address field.

Example: The binary number 0011 will designate register R3.

1. Zero Address Instruction

This instruction does not have an operand field, and the location of operands is implicitly represented. The stack-organized computer system supports these instructions. To evaluate the arithmetic expression, it is required to convert it into reverse polish notation.



Example

In Postfix Notation, $X = (A + B) * (C + D)$ would be written as $AB+CD+*$.

PUSH	A	TOP = A
PUSH	B	TOP = B
ADD		TOP = A+B
PUSH	C	TOP = C
PUSH	D	TOP = D
ADD		TOP = C+D
MUL		TOP = (C+D)*(A+B)
POP	X	M[X] = TOP

COMPUTER ORGANIZATION AND ARCHITECTURE

UNIT –II

TOPIC- TYPES OF INSTRUCTION FORMATS WITH EXAMPLE PART-2

Types of Instruction Formats

Depending on the multiple address fields, the instruction is categorized as follows:

1. **Zero address instruction**
2. **One address instruction**
3. **Two address instruction**
4. **Three address instruction**

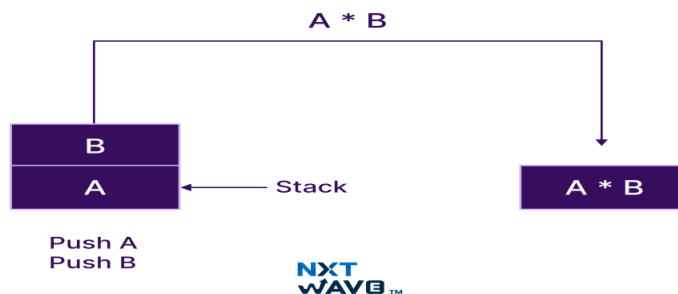
The operations specified by a computer instruction are executed on data stored in memory or processor registers. The operands residing in processor registers are specified with an address. The registered address is a binary number of k bits that defines one of the 2^k registers in the CPU.

Thus, a CPU with 16 processors registers R0 through R15 and will have a four-bit register address field.

Example: The binary number 0011 will designate register R3.

1. Zero Address Instruction

This instruction does not have an operand field, and the location of operands is implicitly represented. The stack-organized computer system supports these instructions. To evaluate the arithmetic expression, it is required to convert it into reverse polish notation.



Example

In Postfix Notation, $X = (A + B) * (C + D)$ would be written as $AB+CD+*$.

PUSH	A	TOP = A
------	---	---------

PUSH	B	TOP = B
ADD		TOP = A+B
PUSH	C	TOP = C
PUSH	D	TOP = D
ADD		TOP = C+D
MUL		TOP = (C+D)*(A+B)
POP	X	M[X] = TOP

2. One Address Instruction

Specifies a single operand, typically held in an accumulator. The accumulator is an implicit register used in the operation. The result can either be stored in the accumulator or another location.

Example

The expression $X = (A + B) * (C + D)$ can be computed using one-address instructions with the accumulator (AC).

opcode	operand/ address of operand	mode
--------	-----------------------------------	------

LOAD	A	AC = M[A]
ADD	B	AC = AC + M[B]
STORE	T	M[T] = AC
LOAD	C	AC = M[C]

ADD	D	$AC = AC + M[D]$
MUL	T	$AC = AC * M[T]$
STORE	X	$M[X] = AC$

All operations are done between the accumulator(AC) register and a memory operand.
 $M[]$ is any memory location.
 $M[T]$ addresses a temporary memory location for storing the intermediate result.

This instruction format has only one operand field.

This address field uses two special instructions to perform data transfer, namely:

- **LOAD:** This is used to transfer the data to the accumulator.
- **STORE:** This is used to move the data from the accumulator to the memory.

3. Two Address Instructions

It allows for two operands to be specified. These operands can be registers or memory locations. The result can overwrite one of the source operands or can be stored in a separate location

Example

The expression $X = (A + B) * (C + D)$ can be calculated using two registers.

opcode	Destination Address	Source Address	mode
--------	---------------------	----------------	------

MOV	R1, A	$R1 = M[A]$
ADD	R1, B	$R1 = R1 + M[B]$
MOV	R2, C	$R2 = M[C]$
ADD	R2, D	$R2 = R2 + M[D]$
MUL	R1, R2	$R1 = R1 * R2$
MOV	X, R1	$M[X] = R1$

The MOV instruction transfers the operands to the memory from the processor registers. R1, R2 registers.

4. Three Address Instructions

It specifies three operands contains two source operands and one destination operand. This format allows for more complex operations and direct representation of expressions. It is most commonly used in high-level language compilers and optimizers.

Example

The expression $X = (A + B) * (C + D)$ can be computed with three addresses.

opcode	Destination address	Source address	Source address	mode
--------	---------------------	----------------	----------------	------

ADD	R1, A, B	$R1 = M[A] + M[B]$		
ADD	R2, C, D	$R2 = M[C] + M[D]$		
MUL	X, R1, R2	$M[X] = R1 * R2$		

Advantages of Instruction Formats

- Allows efficient processing of instructions.
- Easier hardware design with consistent instruction structure.
- Simple structure speeds up instruction decoding.
- Simplifies code generation for better performance.
- Accommodates a wide range of instructions

Disadvantages of Instruction Formats

- Balancing flexibility and efficiency can be difficult.
- Complex formats can slow down decoding.
- More complex formats may reduce speed.
- May not scale with new technologies.
- More bits can waste memory and bandwidth

COMPUTER ORGANIZATION AND ARCHITECTURE**UNIT –II****TOPIC- ADDRESSING MODES ,****TYPES OF ADDRESSING MODES WITH EXAMPLE PART-1****Addressing Modes**

Addressing modes are an aspect of the instruction set architecture in most CPU design. The term addressing modes refers to the way in which the operand of an instruction is specified. The various addressing modes helps to identify the types of operands in the instruction. An addressing mode specifies how to calculate the effective memory address of an operand.

Generally, the programs are written in a high-level language and later compiled to generate the machine code. The machine code has low-level instructions. The low-level instruction has opcode and operands. Addressing mode has nothing to do with the opcode part. It focuses on presenting the operand's address in the instructions.

Why are Addressing Modes Used?

Addressing modes shows the location of a required object in the computer. The object may be an instruction or data. The output of the addressing mode is an effective address [EA].

Effective Address (EA)

The effective address refers to the address of an exact memory location in which an operand's value is actually present.

The addressing mode is implemented in the instruction in two ways:

- Implicit: The opcode itself specifies the addressing mode used.
- Explicit: The mode field is used in the instruction format to specify the type of addressing mode used.

Types of Addressing modes

Different ways of specifying the location of an operand in an instruction are called as addressing modes which are as follows:

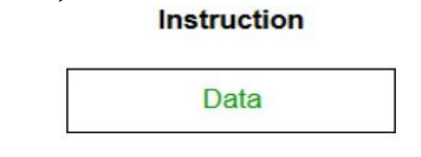
1. Implied/Implicit Addressing Mode
2. Immediate Addressing Mode
3. Direct/Absolute Addressing Mode
4. Indirect Addressing Mode
5. Register Direct Addressing Mode
6. Register Indirect Addressing Mode
7. Relative Addressing Mode

8. Indexed Addressing Mode
9. Base Register Addressing Mode
10. Auto-increment Addressing Mode
11. Auto-decrement Addressing Mode

1. Implied Addressing Mode: In this addressing mode, the definition of the instruction itself specify the operands implicitly. It is also called as implicit addressing mode.

e.g.

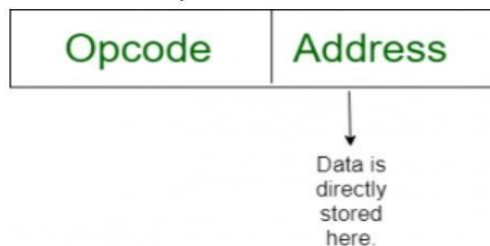
- The instruction “Complement Accumulator” is an implied mode instruction (CMA).



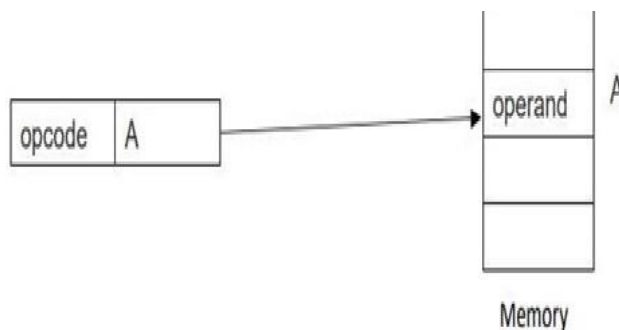
2. Immediate Addressing Mode: In this addressing mode, the operand is specified in the instruction explicitly. Instead of address field, an operand field is present that contains the operand.

e.g. ADD 100 will increment the value stored in the accumulator by 100

MOV R #20 initialized register R to a constant value 20.

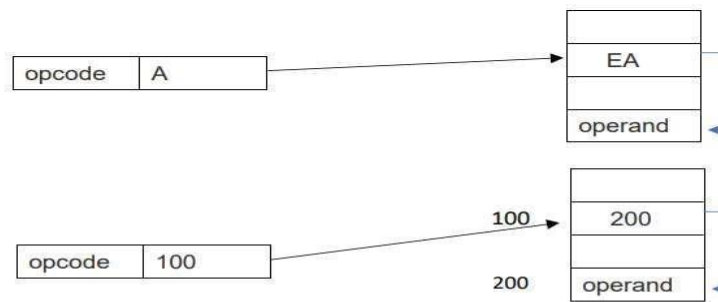


3. Direct Addressing Mode: In this addressing mode, the address field of the instruction contains the effective address of the operand. Only one reference to memory is required to fetch the operand. It is also called as absolute addressing mode.



4. Indirect Addressing Mode: In this addressing mode, the address field of the

instruction specifies the address of memory location that contains the effective address of the operand. Two references to memory are required to fetch the



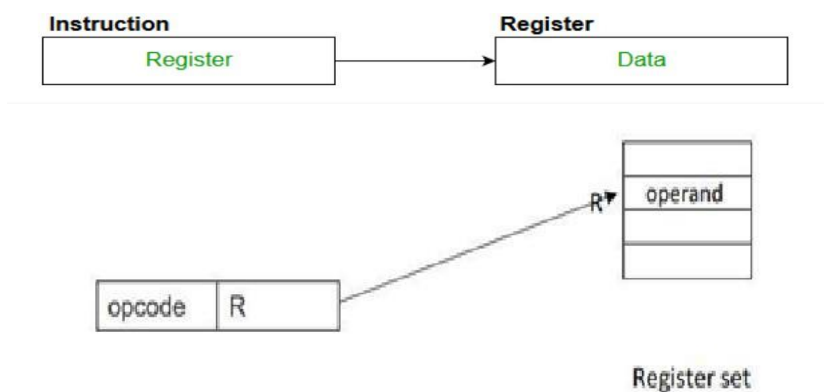
Example-

ADD X will increment the value stored in the accumulator by the value stored at memory location specified by X.

$$AC \leftarrow AC + [[X]]$$

operand.

5. Register Direct Addressing Mode: In this addressing mode, the operand is contained in a register set. The address field of the instruction refers to a CPU register that contains the operand. No reference to memory is required to fetch the operand.



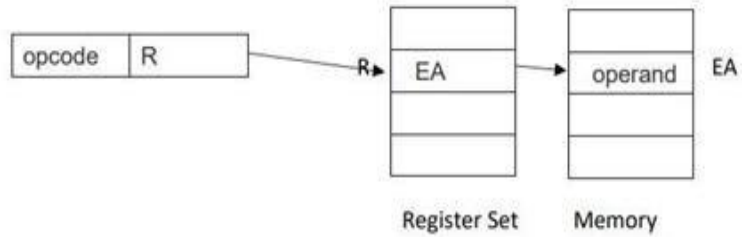
Example-

ADD R will increment the value stored in the accumulator by the content of register R.

$$AC \leftarrow AC + [R]$$

6. Register Indirect Addressing Mode: In this addressing mode, the address field of the instruction refers to a CPU register that contains the effective address of the operand. Only one reference to memory is required to fetch the operand.





Example-

ADD R will increment the value stored in the accumulator by the content of memory location specified in register R.

$AC \leftarrow AC + [[R]]$

COMPUTER ORGANIZATION AND ARCHITECTURE**UNIT –II****TOPIC- TYPES OF ADDRESSING MODES WITH EXAMPLE PART-2****Addressing Modes**

Addressing modes are an aspect of the instruction set architecture in most CPU design. The term addressing modes refers to the way in which the operand of an instruction is specified. The various addressing modes helps to identify the types of operands in the instruction. An addressing mode specifies how to calculate the effective memory address of an operand.

Generally, the programs are written in a high-level language and later compiled to generate the machine code. The machine code has low-level instructions. The low-level instruction has opcode and operands. Addressing mode has nothing to do with the opcode part. It focuses on presenting the operand's address in the instructions.

Types of Addressing Modes

Different ways of specifying the location of an operand in an instruction are called as addressing modes which are as follows:

1. Implied/Implicit Addressing Mode
2. Immediate Addressing Mode
3. Direct/Absolute Addressing Mode
4. Indirect Addressing Mode
5. Register Direct Addressing Mode
6. Register Indirect Addressing Mode
7. Relative Addressing Mode
8. Indexed Addressing Mode
9. Base Register Addressing Mode
10. Auto-increment Addressing Mode
11. Auto-decrement Addressing Mode

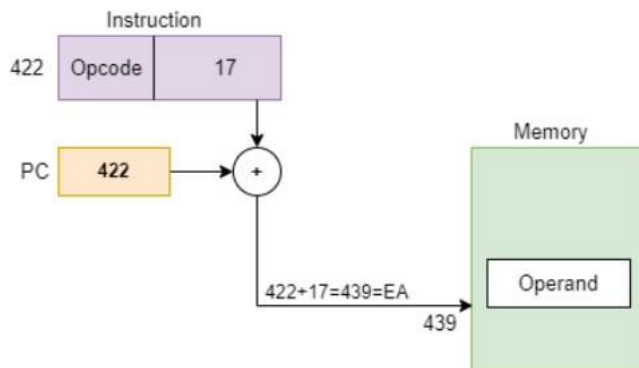
7. Relative Addressing Mode

This mode is another version of the displacement address mode. The program counter's content is added to the instruction's address part to obtain the effective address.

$$EA = A + (PC)$$

Here, EA: Effective address, PC: program counter.

PC: It contains the address of the next instruction to be executed.

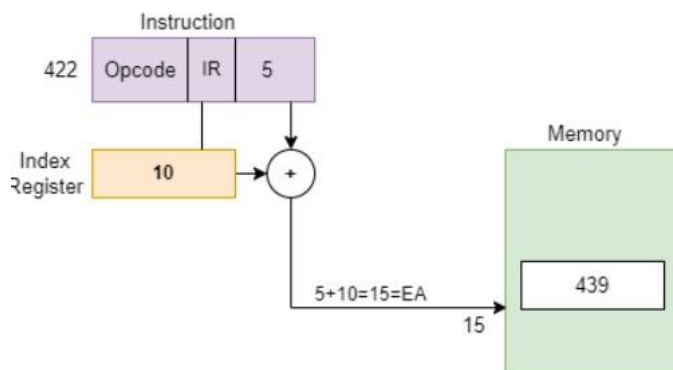


The program counter contains the number 422, and the address part of the instruction contains the number 17. The instruction at location 421 is read during the fetch phase, and the program counter is then incremented by one to $422 + 17 = 439$.

8. Indexed Addressing Mode

The index register's content is added to the instruction's address to obtain the effective address.

$EA = \text{content of index register} + \text{Instruction address part}$

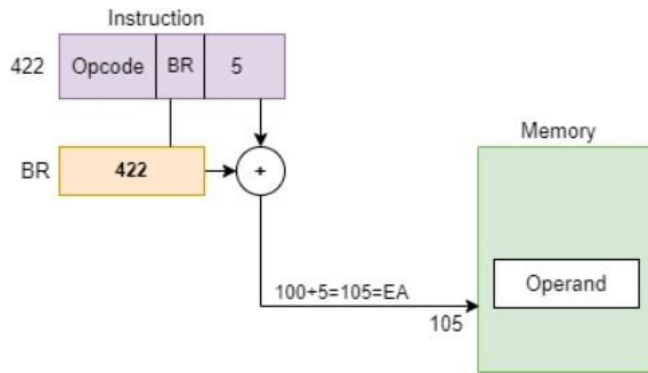


9. Base Register Addressing Mode

This mode is another version of the displacement address mode. To obtain the effective address, the base register's content is added to the instruction's address.

$EA = A + (R)$

A: Displacement, R: Pointer to the base address.



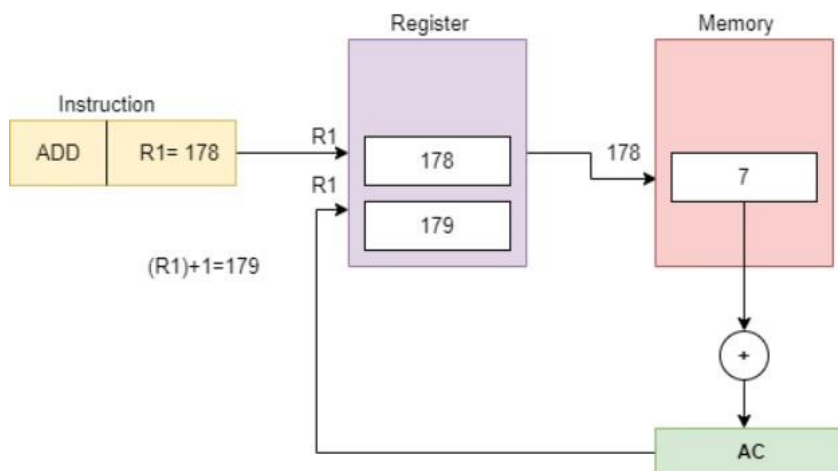
10. Auto-Increment Addressing Mode

This mode is a special Register Indirect Addressing Mode case. Effective address of the operand is the contents of a register specified in the instruction.

After accessing the operand, the contents of this register are automatically incremented to point to the next consecutive memory location.

(R1)+.

Here one register reference, one memory reference are required to fetch the operand



Here, the Effective Address (R) =178, and the operand in AC are 7. After loading R1 is incremented by 1 and becomes 179.

11. Auto-Decrement Addressing Mode

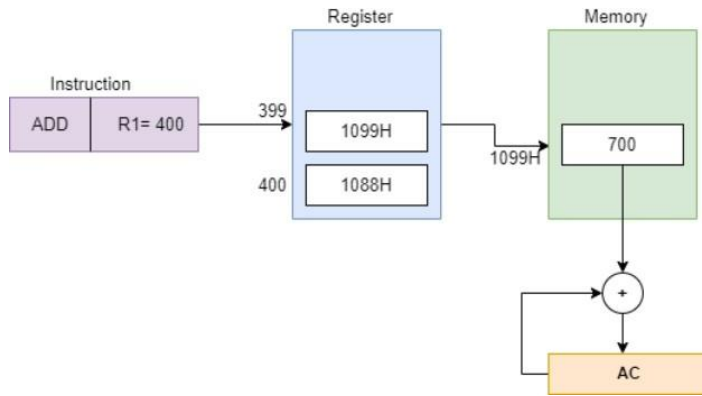
This mode is also a special Register Indirect Addressing Mode case.

Effective address of the operand is the contents of a register specified in the instruction.

Before accessing the operand, the contents of this register are automatically decremented to point to the previous consecutive memory location.

-(R1)

Here one register reference, one memory reference are required to fetch the operand.



Here, register R1 is decremented by 1 ($400-1=399$) prior to the instruction execution which implies that the operand loaded to the AC is of address 1099H instead of 1088H. Hence, the Effective Address is 1099H.

Numerical Example:

PC = 200

R1 = 400

XR = 100

AC

Address	Memory
200	Load to AC Mode
201	Address = 500
202	Next instruction
399	450
400	700
500	800
600	900
702	325
800	300

Addressing Mode	Effective Address		Content of AC
Direct address	500	$/* AC \leftarrow (500)$	$*/ 800$
Immediate operand	-	$/* AC \leftarrow 500$	$*/ 500$
Indirect address	800	$/* AC \leftarrow ((500))$	$*/ 300$
Relative address	702	$/* AC \leftarrow (PC+500)$	$*/ 325$
Indexed address	600	$/* AC \leftarrow (XR+500)$	$*/ 900$
Register	-	$/* AC \leftarrow R1$	$*/ 400$
Register indirect	400	$/* AC \leftarrow (R1)$	$*/ 700$
Autoincrement	400	$/* AC \leftarrow (R1)+$	$*/ 700$
Autodecrement	399	$/* AC \leftarrow -(R)$	$*/ 450$

COMPUTER ORGANIZATION AND ARCHITECTURE**UNIT –II****TOPIC- DATA TRANSFER INSTRUCTIONS****Data Transfer Instructions**

Computer provides an extensive set of instructions to give the user the flexibility to carry out various computational tasks. Most computer instruction can be classified into three categories:

- (1) Data transfer instruction.
- (2) Data manipulation instruction.
- (3) Program control instruction.

Data transfer involves the movement of data between different components of a computer system, such as from the processor to memory, between registers, or across network interfaces. This movement is essential for fetching instructions, storing results, and communicating between various parts of a system.

Data transfer instruction causes transferring data from one location to another without changing the binary instruction content.

Data manipulation instructions are those that perform arithmetic logic, and shift operations.

Program control instructions provide decision-making capabilities and change the path taken by the program when executed in the computer.

There are eight commonly used data transfer instructions represented by a mnemonic symbol shown in below table:

Name	Mnemonic Symbols
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	In
Output	OUT
Push	PUSH
Pop	POP

- **Load** – The load instruction is used to transfer data from the memory to a processor register, which is usually an accumulator.

Ex: LDR R1, [R2] loads the contents of the memory location pointed by R2 into register R1.

- **Store** – The store instruction transfers data from processor registers to memory.

Ex: STR R3, [R4] stores the contents of register R3 into the memory location pointed by R4.

- **Move** – The move instruction transfers data from processor register to memory or memory to processor register or between processor registers itself. The source can be a register, a memory location, or an immediate number.

Ex: MOV AX, BX copies the contents of register BX to register AX.

MOV [SI], CX ; Copy the contents of register CX to the memory location pointed by SI
MOV AX, 5000H
MOV AX, [SI]
MOV AX, [2000H]
MOV AX, 50H[BX]
MOV CL, [357AH]

- **Exchange** – The exchange instruction swaps information either between two registers or between a register and a memory word.

Ex: XCHG [2050], AX
XCHG [5000H], AX
XCHG BX, AX

- **Input** – The input instruction transfers data between the processor register and the input terminal.

Ex: IN AL, 03H
IN AX, DX

- **Output** – The output instruction transfers data between the processor register and the output terminal.

Ex: OUT 03H, AL
OUT DX, AX

- **Push and Pop** – The push and pop instructions transfer data between a processor register and memory stack.

Ex: PUSH AX pushes the contents of register AX onto the stack.
POP BX pops the topmost value from the stack into register BX.

As an example, consider the load to accumulator instruction when used with eight different addressing modes.

MODE	ASSEMBLY CONVENTION	REGISTER TRANSFER
Direct Address	LD ADR	$AC \leftarrow M[ADR]$
Indirect Address	LD @ADR	$AC \leftarrow M[M[ADR]]$
Relative Address	LD \$ADR	$AC \leftarrow M[PC+ADR]$
Immediate Address	LD #NBR	$AC \leftarrow NBR$
Index Address	LD ADR(X)	$AC \leftarrow M[ADR+XR]$
Register	LD R1	$AC \leftarrow R1$
Register Indirect	LD (R1)	$AC \leftarrow M[R1]$
Autoincrement	LD (R1)+	$AC \leftarrow M[R1], R1 \leftarrow R1+1$

ADR stands for an address.

NBA a number or operand.

X is an index register.

R1 is a processor register

AC is the accumulator register.

The @ character symbolizes an indirect addressing.

The \$ character before an address makes the address relative to the program counter PC.

The # character precedes the operand in an immediate-mode instruction.

An indexed mode instruction is recognized by a register that placed in parentheses after the symbolic address. The register mode is symbolized by giving the name of a processor register.

Advantages of Data Transfer Instructions

- **Faster Operations:** Data transfers proceed considerably more quickly, saving time on chores like backups and migrations.
- **Less Memory Usage:** They utilize less memory, which frees up resources for other operations.
- **Improved Communication:** They facilitate improved resource utilization by enhancing communication between various systems and applications.

- **Simple Big Data Handling:** They move massive amounts of data without manual labor, perfect for major processes involving several systems.
- **Accuracy:** Data transfers are more exact, preventing mistakes in transactions involving sensitive data or money.
- **Easier Coding:** Streamlining the coding procedure minimizes complexity and shortens the time required to write code.

Disadvantages of Data Transfer Instructions

- **Resource use:** Transferring data depletes memory and processing capacity, especially when working with huge amounts of data. The system's efficiency may suffer as a result.
- **Bandwidth restrictions:** Transferring vast volumes of data can be difficult or impossible if the bandwidth between components is constrained or unreliable.
- **Security hazard:** Network congestion can prolong transmission times and raise the chance of sensitive data exposure.

COMPUTER ORGANIZATION AND ARCHITECTURE
UNIT –II
TOPIC- DATA MANIPULATION INSTRUCTIONS PART-1

Data Manipulation Instructions

Data manipulation refers to the processes that transform and operate on data to produce desired outcomes. This includes arithmetic operations, logical operations, bit-level manipulations, and data shifting. Each of these operations plays a vital role in how computers process information and execute tasks.

Data manipulation instructions can be categorized into three parts:

- 1) Arithmetic instructions
- 2) Logical and bit manipulation instructions
- 3) Shift instructions

1. Arithmetic Instructions

Arithmetic instructions include increment, decrement, add, subtract, multiply, divide, add with Carry, subtract with Borrow, negate that is (2's) two's complement. If there's a negative number, it is considered as negate (so two's complement).

The table given below shows the Arithmetic Instructions:

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

Name	Mnemonic	Example	Explanation
Increment	INC	INC R1	Increases the value stored in register R1 by 1.
Decrement	DEC	DEC R2	Decreases the value stored in register R2 by 1.
Add	ADD	ADD R3, R4	Adds the value in register R4 to the value in

Name	Mnemonic	Example	Explanation
			register R3.
Subtract	SUB	SUB R5, R6	Subtracts the value in register R6 from the value in R5.
Multiply	MUL	MUL R7, R8	Multiplies the value in register R7 by the value in R8.
Divide	DIV	DIV R9, R10	Divides the value in register R9 by the value in R10.
Add with carry	ADDC	ADDC R11, R12	Adds the value in register R12 and the carry flag to R11.
Subtract with borrow	SUBB	SUBB R13, R14	Subtracts the value in R14 and the borrow flag from R13.

2. Logical and Bit manipulation Instructions

Logical instructions consider each operand bit individually and treat it as a Boolean variable. Basically, logical instructions help perform binary operations on strings of bits stored in registers.

- Clear instruction means making all the bits of a register '0'.
- AND instruction is sometimes referred to as bit clear instruction or mask.
- OR instruction is sometimes referred to as bit set instruction.
- Set instruction means making all the bits of a register '1'.
- XOR instruction is referred to as bit complement instruction.

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

Name	Mnemonic	Example	Explanation
Clear	CLR	CLR R1	Sets the value in register R1 to 0.
Complement	COM	COM R2	Inverts all the bits in the value stored in register R2.
AND	AND	AND R3, R4	Performs a bitwise AND between values in R3 and R4, stores the result in R3.
OR	OR	OR R5, R6	Performs a bitwise OR between values in R5 and R6, stores the result in R5.
Exclusive-OR	XOR	XOR R7, R8	Performs a bitwise XOR between values in R7 and R8, stores the result in R7.
Clear carry	CLRC	CLRC	Clears the carry flag (sets it to 0).
Set Carry	SETC	SETC	Sets the carry flag to 1.
Complement Carry	COMC	COMC	Inverts the carry flag (if it was 1, it becomes 0, and vice versa).
Enable Interrupt	EI	EI	Enables the interrupt system, allowing interrupts to occur.
Disable Interrupt	DI	DI	Disables the interrupt system, preventing interrupts from occurring.

COMPUTER ORGANIZATION AND ARCHITECTURE
UNIT –II
TOPIC- DATA MANIPULATION INSTRUCTIONS PART-2

Data Manipulation Instructions

Data manipulation refers to the processes that transform and operate on data to produce desired outcomes. This includes arithmetic operations, logical operations, bit-level manipulations, and data shifting. Each of these operations plays a vital role in how computers process information and execute tasks.

Data manipulation instructions can be categorized into three parts:

- 1) **Arithmetic instructions**
- 2) **Logical and bit manipulation instructions**
- 3) **Shift instructions**

Shift Instructions

Shift instructions allow the bits of a memory byte or register to be shifted one-bit place to the right or the Left.

There are basically two types:

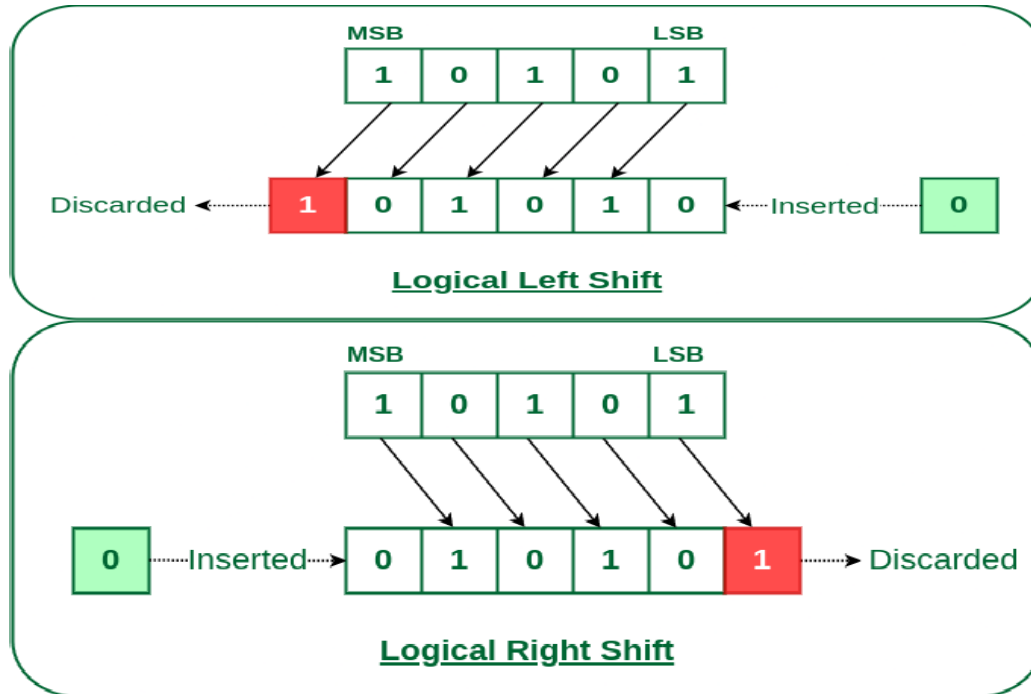
1. Arithmetic shift
 2. Logical shift.
-
1. Arithmetic shifts consider the contents of the memory byte or register to be a signed number. So, when the shift is made, the number is arithmetically divided by two (right shift) or multiplied by two (left shift).
 2. Logical shifts consider the contents of the register or memory byte to be just a bit pattern when the shift is made.

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Logical shift:

In logical left shift, moves each bit to the left one by one. The Empty least significant bit (LSB) is filled with zero (i.e, the serial input), and the most significant bit (MSB) is rejected.

In logical right shift, each bit moves to the right one by one and the least significant bit(LSB) is rejected and the empty MSB is filled with zero.

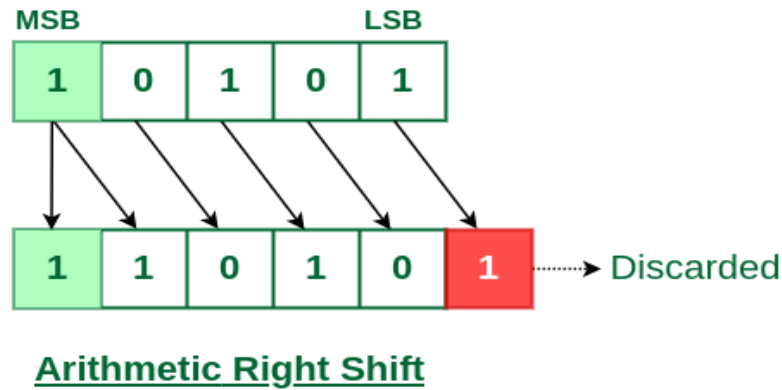
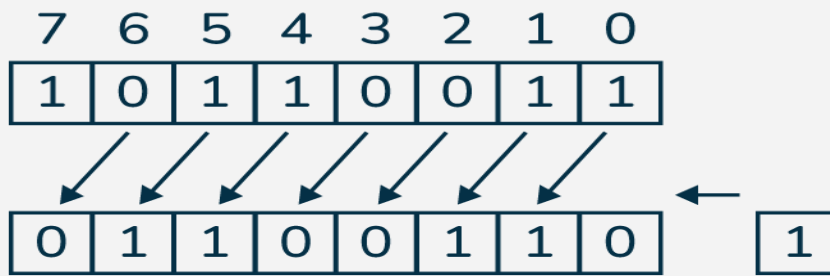


Arithmetic Shift: An arithmetic shift is a microoperation that shifts a signed binary number to the left or right. An arithmetic shift-left multiplies a signed binary number by 2. An arithmetic shift-right divides the number by 2. Arithmetic shifts must leave the sign bit unchanged because the sign of the number remains the same when it is multiplied or divided by 2.

In Arithmetic left shift, each bit is moved to the left one by one. The empty least significant bit (LSB) is filled with zero and the most significant bit (MSB) is rejected. It is same as the Left Logical Shift.

In Arithmetic right shift, each bit is moved to the right one by one and the least significant(LSB) bit is rejected and the empty most significant bit(MSB) is filled with the value of the previous MSB.

Shift Left Arithmetic



Rotate

In rotate instructions, bits of operand are rotated either to the left or right side. Rotate instructions produces a circular shift. Bits shifted out at one end of the word are not lost as in a logical shift but are circulated back into the other end.

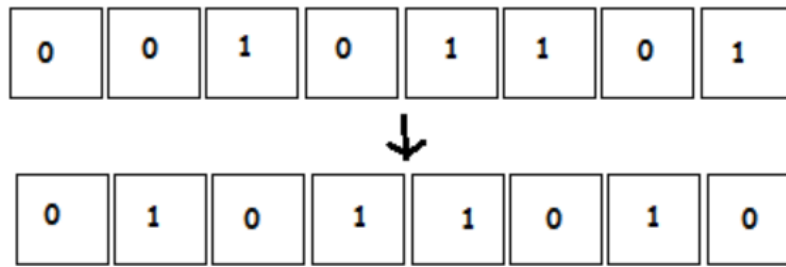
- *ROL (Rotate Left) Instruction,*
- *ROR (Rotate Right) Instruction,*
- *RCL (Rotate Carry Left) Instruction,*
- *RCR (Rotate Carry Right) Rotate Carry Right.*

The right rotate instruction shifts all bits in the register or memory operand specified. The carry flag (CF) is included in the rotation.

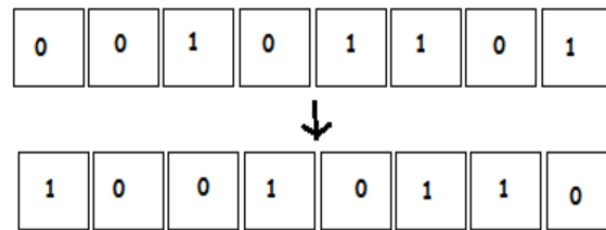
The least significant bit is rotated to the carry flag, the carry flag is rotated to the most significant bit position, all other bits are shifted to the right. The result includes the original value of the carry flag.

The left rotate instruction shifts all bits in the register or memory operand specified. The carry flag (CF) is included in the rotation.

The most significant bit is rotated to the carry flag, the carry flag is rotated to the least significant bit position, all other bits are shifted to the left.

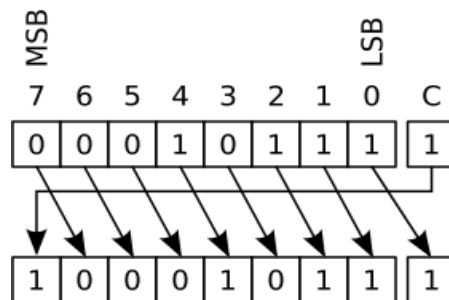


Left Rotate

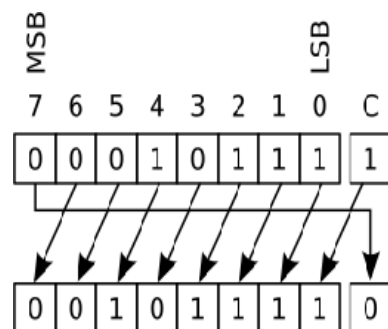


RIGHT ROTATION EXAMPLE

- The RCL (Rotate and Carry Left) instruction shifts each bit to the left, copies the Carry flag to the least significant bit and copies the most significant bit into the Carry flag.
- The RCR (Rotate and Carry Right) instruction shifts each bit to the right, copies the Carry flag to the most significant bit and copies the least significant bit into the Carry flag.



Rotate right through carry



Rotate left through carry

COMPUTER ORGANIZATION AND ARCHITECTURE**UNIT –II****TOPIC- PROGRAM CONTROL INSTRUCTIONS****Program Control Instructions**

Instructions are always stored in successive memory locations. When processed in the CPU, the instructions are fetched from consecutive memory locations and executed. Each time an instruction is fetched from memory, the program counter is incremented so that it contains the address of the next instruction in sequence.

A program control type of instruction, when executed, may change the address value in the program counter and cause the flow of control to be altered.

In other words, program control instructions specify conditions for altering the content of the program counter, while data transfer and manipulation instructions specify conditions for data-processing operations. The change in value of the program counter as a result of the execution of a program control instruction causes a break in the sequence of instruction execution.

This is an important feature in digital computers, as it provides control over the flow of program execution and a capability for branching to different program segments.

Some typical program control instructions are listed in Table below:

Name	Mnemonics
Branch	BR
Jump	JMP
Skip	SKP
Call	Call
Return	RET
Compare (by Subtraction)	CMP
Test (by ANDing)	TST

Branch and Jump

The branch is usually a one-address instruction. It is written in assembly language as BR ADR, where ADR is a symbolic name for an address. When executed, the branch instruction causes a transfer of the value of ADR into the program counter. Since the program counter contains the address of the instruction to be executed, the next instruction will come from location ADR

Branch and jump instructions may be conditional or unconditional.

An unconditional branch instruction causes a branch to the specified address without any conditions. The conditional branch instruction specifies a condition such as branch if positive or branch if zero. If the condition is met, the program counter is loaded with the branch address and the next instruction is taken from this address. If the condition is not met, the program counter is not changed and the next instruction is taken from the next location in sequence.

Conditional Branch Instructions

Mnemonic	Branch condition	Tested condition
BZ	Branch if zero	$Z = 1$
BNZ	Branch if not zero	$Z = 0$
BC	Branch if carry	$C = 1$
BNC	Branch if no carry	$C = 0$
BP	Branch if plus	$S = 0$
BM	Branch if minus	$S = 1$
BV	Branch if overflow	$V = 1$
BNV	Branch if no overflow	$V = 0$
<i>Unsigned compare conditions (A - B)</i>		
BHI	Branch if higher	$A > B$
BHE	Branch if higher or equal	$A \geq B$
BLO	Branch if lower	$A < B$
BLOE	Branch if lower or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$
<i>Signed compare conditions (A - B)</i>		
BGT	Branch if greater than	$A > B$
BGE	Branch if greater or equal	$A \geq B$
BLT	Branch if less than	$A < B$
BLE	Branch if less or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

Table 8-11

37

Skip

The skip instruction does not need an address field and is therefore a zero-address instruction. A conditional skip instruction will skip the next instruction if the condition is met. This is accomplished by incrementing the program counter during the execute phase in addition to its being incremented during the fetch phase.

If the condition is not met, control proceeds with the next instruction in sequence where the programmer inserts an unconditional branch instruction. Thus a skip-branch pair of instructions causes a branch if the condition is not met, while a single conditional branch instruction causes a branch if the condition is met.

Call and Return

A subroutine is a self-contained sequence of instructions that performs a given computational task. During the execution of a program, a subroutine may be called to perform its function many times at various points in the main program. Each time a subroutine is called, a branch is executed to the beginning of the subroutine to start executing its set of instructions. After the subroutine has been executed, a branch is made back to the main program. The instruction that transfers program control to a subroutine is known by different names. The most common names used are call subroutine, jump to subroutine, branch to subroutine, or branch and save address.

A call subroutine instruction consists of an operation code together with an address that specifies the beginning of the subroutine.

The instruction is executed by performing two operations:

- (1) the address of the next instruction available in the program counter (the return address) is stored in a temporary location so the subroutine knows where to return, and
- (2) control is transferred to the beginning of the subroutine. The last instruction of every subroutine, commonly called return from subroutine, transfers the return address from the temporary location into the program counter. This results in a transfer of program control to the instruction whose address was originally stored in the temporary location.

Different computers use a different temporary location for storing the return address. Some store the return address in the first memory location of the subroutine, some store it in a fixed location in memory, some store it in a processor register, and some store it in a memory stack. The most efficient way is to store the return address in a memory stack.

A subroutine call is implemented with the following microoperations:

SP $\leftarrow$ SP - 1
M [SP] $\leftarrow$ PC
PC $\leftarrow$ effective address
Decrement stack pointer
Push content of PC onto the stack

A subroutine return is implemented with the following microoperations:

PC $\leftarrow$ M[SP]
SP $\leftarrow$ SP + 1
Pop stack and transfer to PC
Increment stack pointer
By using a subroutine stack, all return addresses are automatically stored by the hardware in one unit.

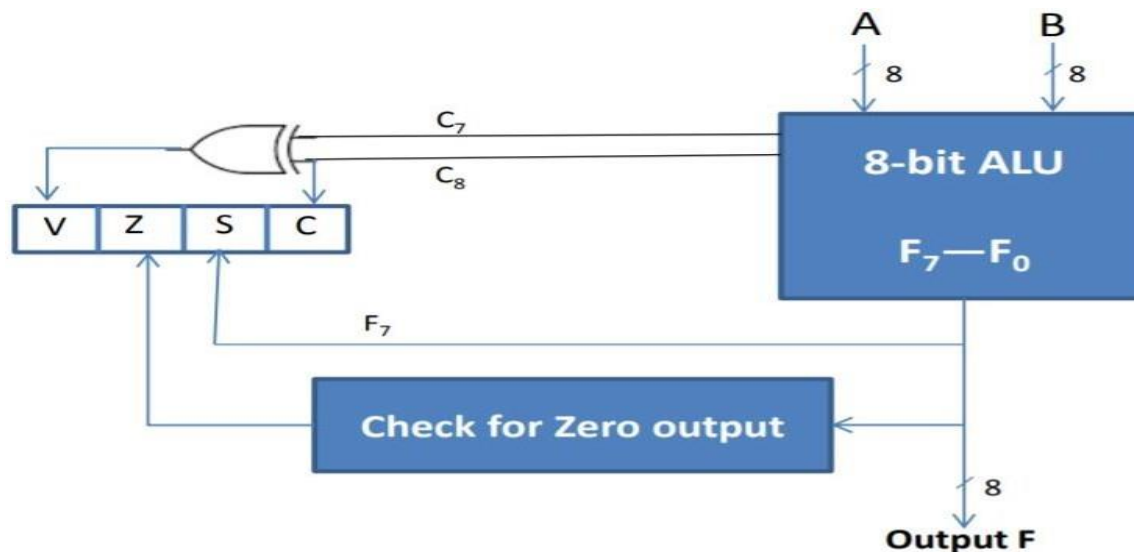
Compare and Test

The compare and test instructions do not change the program sequence directly. The compare instruction performs a subtraction between two operands, but the result of the operation is not retained. However, Certain status bit conditions are set as a result of the operation.

Similarly, the test instruction performs the logical AND of two operands and updates certain status bits without retaining the result or changing the operands. The status bits of interest are the carry bit, the sign bit, a zero indication, and an overflow condition. The generation of these status bits will be discussed

Status Bit Conditions: It is sometimes convenient to supplement the ALU circuit in the CPU with a status register where status bit conditions can be stored for further analysis. Status bits are also called condition-code bits or flag bits. Figure below shows the block diagram of an 8-bit ALU with a 4-bit status register.

The four status bits are symbolized by C, S, Z, and V. The bits are set or cleared as a result of an operation performed in the ALU.



1. Bit C (carry) is set to 1 if the end carry C₈ is 1. It is cleared to 0 if the carry is 0.
2. Bit S (sign) is set to 1 if the highest-order bit F₇ is 1. It is set to 0 if the bit is 0.
3. Bit Z (zero) is set to 1 if the output of the ALU contains all 0's. It is cleared to 0 otherwise. In other words, Z = 1 if the output is zero and Z = 0 if the output is not zero.
4. Bit V (overflow) is set to 1 if the exclusive-OR of the last two carries is equal to 1, and cleared to 0 otherwise.

For the 8-bit ALU, V = 1 if the output is greater than + 127 or less than - 128.

The status bits can be checked after an ALU operation to determine certain relationships that exist between the values of A and B. If bit V is set after the addition of two signed numbers, it indicates an overflow condition. If Z is set after an exclusive-OR operation, it indicates that A = B. This is so because the exclusive-OR of two equal operands gives an all-0's result which sets the Z bit.

A single bit in A can be checked to determine if it is 0 or 1 by masking all bits except the bit in question and then checking the Z status bit. For example, let A = 101x1100, where x is the bit to be checked. The AND operation of A with B = 00010000 produces a result 000x0000. If x = 0, the Z status bit is set, but if x = 1, the Z bit is cleared since the result is not zero.